

Project Proposal
Cloud-Based IoT Environmental Telemetry Platform
Seid Cubro - Charles Shoppel

1. Summary:

This project proposes the design and implementation of a Cloud-Based IoT Environmental Telemetry Platform that ingests temperature and humidity data from IoT devices, processes the data using cloud-native microservices, and exposes the data through secure REST APIs.

The system will be built using containerized services, Infrastructure as Code, CI/CD automation, observability tooling, and security-first design principles. The project follows a structured, milestone-driven lifecycle aligned with industry-standard cloud engineering practices and includes design reviews, traceability, deployment, testing, and operational readiness validation.

2. Project Scope:

- Cloud-native ingestion and API microservices
- Containerized deployment
- Infrastructure provisioning using Infrastructure as Code
- CI/CD pipeline from commit to deployment
- Centralized logging, metrics, and alerting
- Authentication, authorization, and secrets management
- Kubernetes-based deployment with autoscaling
- Asynchronous ingestion using queues/streams
- Load testing, tuning, and cost controls
- Operational readiness and demo

3. Functional Requirements (FRs):

- FR-1 The system shall ingest temperature and humidity data via an API or message stream
- FR-2 The system shall persist telemetry data in a cloud database
- FR-3 The system shall expose APIs to retrieve latest and historical telemetry
- FR-4 The system shall deploy ingestion and API services as independent containers
- FR-5 The system shall authenticate API requests
- FR-6 The system shall process ingestion asynchronously using a queue or stream
- FR-7 The system shall expose health endpoints for services
- FR-8 The system shall deploy to Kubernetes with ingress and service discovery
- FR-9 The system shall autoscale services based on load
- FR-10 The system shall support rollback to previous deployments

4. Non-Functional Requirements (NFRs):

- NFR-1 Infrastructure must be reproducible via Infrastructure as Code
- NFR-2 The system must tolerate individual service failures
- NFR-3 Secrets must be securely managed and never stored in code
- NFR-4 Logs and metrics must be centrally collected
- NFR-5 Alerts must be generated for failures or threshold breaches
- NFR-6 The system must operate within defined cost controls
- NFR-7 The system must support operational readiness (backup, rollback, runbook)

5. Success Criteria:

The project will be considered successful if:

- All FRs and NFRs are implemented and traceable
- The system deploys reproducibly from scratch
- CI/CD pipelines function end-to-end
- Kubernetes autoscaling and async ingestion operate correctly
- Observability and alerts detect failure scenarios
- The system withstands load testing within constraints
- A complete demo and documentation are delivered

6. Backlog Table:

ID	Task	Target	Owner	Done When...
B-1	Create project repository and starter README	M2	Application/DevOps	Repo exists with README and local run steps
B-2	Define architecture diagram and component responsibilities	M3	Project Lead	Diagram and design document approved
B-3	Implement local ingestion API prototype	M4	Application/DevOps	POST endpoint returns 202
B-4	Implement local query API prototype	M4	Application/DevOps	GET endpoint returns 200
B-5	Establish observability and security baseline	M5	Project Lead	Logging, health checks, auth approach defined

B-6	Provision cloud infrastructure and deploy to Kubernetes	M6	Project Lead	Cloud resources provisioned; services reachable
B-7	Integrate asynchronous processing (SQS + worker)	M7	Project Lead	Telemetry processed end-to-end asynchronously
B-8	Execute load testing and cost controls	M8	Project Lead	Bottlenecks identified, cost controls verified
B-9	Prepare and deliver team demo	M9	Team	Live demo complete
B-10	Final documentation and individual reports	M10	Team	Final repo and reports submitted

7. Repository and Starter Documentation:

Repository: <https://github.com/seidcubro/cs361-iot-cloud-telemetry-platform>

The project repository contains a starter README that documents the system overview, architecture summary, technology stack, API endpoints, security model, observability approach, CI/CD workflow and milestone alignment. The README provides reviewers and collaborators with a clear understanding of the project's cloud-native design, asynchronous data flow and operational goals without requiring prior context.

The repository is structured to support incremental development across milestones, with documentation centralized in the /docs directory and application, infrastructure and deployment artifacts added progressively. The README serves as the authoritative entry point for understanding the system's purpose, architecture and development status. Detailed design artifacts, diagrams and review materials are maintained alongside the codebase as the project evolves.

8. Requirements Traceability Table:

Requirement	Component(s)	Verification Method
FR-1 Ingest Telemetry	API Gateway, Ingestion Service	Post request demo
FR2- Persist data	DynamoDB	Data retrieval demo
FR-3 Query telemetry	API Service	GET request demo
FR-4 Containerized services	Docker/Kubernetes	Container runtime evidence
FR-6 Async processing	SQS + Worker	Queue backlog + logs
FR-7 Health endpoints	All services	/health endpoint
FR-8 Kubernetes deployment	EKS	Cluster deployment proof
FR-9 Autoscaling	Kubernetes HPA	Load test observation
NFR-3 Secrets management	Secrets Manager	Config review
NFR-4 Observability	CloudWatch	Logs and metrics review

9. Conclusion:

This project demonstrates a full cloud-native system lifecycle, from proposal and design through deployment, testing, and operational readiness. By aligning each phase to industry-standard milestones and deliverables, the project provides both academic rigor and practical cloud engineering experience.